you can disable it on a per-variable basis using the *unsafe* modifier. If you replace *int *pPtr* in *pointertest.c* with *int * unsafe pPtr*, then EiC proceeds without any errors at *pPtr[79]*. The *unsafe* feature is useful when you interact with external, compiled C components.

CINT

Now we come to the heavyweight among all these interpreters. CINT is the C and C++ code interpretation component of the object-oriented data analysis package ROOT, and can be used as a standalone application too. According to the CINT man page, it covers a whopping 95 per cent of ANSI C, and 90 per cent of C++ features. It also provides various features similar to GDB, for debugging interpreted source code. CINT scripts can communicate with compiled components and external applications.

You can install CINT in two ways. If you don't mind the overhead of many extra components, then just run *sudo apt-get install root-system*. To build and install the latest version from source, first install the *readline* library and sources, then download the latest ROOT source tarball, extract and compile it:

```
sudo apt-get install libreadline-dev
tar -zxvf root-version.source.tar.gz
cd root/cint
./configure
make
```

You need to set executable and library search paths, and a CINT-specific setting, so add the following lines to your *.bashrc*:

```
CINTBASE=<absolute path of cint source directory>
export CINTSYSDIR=$CINTBASE
export PATH=$CINTBASE/bin:$PATH
export LD_LIBRARY_PATH=$CINTBASE/lib:$LD_LIBRARY_PATH
export MANPATH=$CINTBASE/doc:$MANPATH
```

Finally, run *sudo ldconfig* to configure the dynamic linker run-time bindings.

To quickly see CINT in action, save the C++ code shown below as *hellocint.cpp* and run *cint hellocint.cpp*.

```
#include <iostream>
using namespace std;

int main()
{
    cout << endl
         <<" Hello to FLOSS from CINT!!!"
         << endl;
    return 0;
}
```

To interpret C and C++ code in multiple files, you need to provide CINT with a list of source files, in which the last file contains the *main()* function. You can run the stack example presented in the Tcc section with CINT—type *cint stack.c test.c*. In case CINT is unable to find a *main()* function, or encounters any other error, it starts an interactive session.

To use the interpreted code debugging feature of CINT, start interpretation with a breakpoint in a desired function, by appending the option *-b* along with the list of source code files. Now examine and/or change the program execution state through the various debugging options mentioned in the CINT man page (or displayed by entering *h* at the CINT interpreter prompt).

Beyond C and C++ interpretation with EiC and CINT

You can use EiC's features to embed a C code interpreter in an external C application. To run a C source file, it provides a function *EiC_run(int argc, char **argv)*, where *argc* represents the number of arguments passed, and *argv* is an array of strings consisting of C source file names plus other command-line arguments. To run a C code file named *myeic.c*, the code sequence will be:

```
char *argv = {"myeic.c", ...};
int argc   = sizeof(argv)/sizeof(char *);
EiC_run(argc, argv);
```

You can also pass C or preprocessor commands to EiC through the function *EiC_parseString(char *command, ...)*. To build applications using these functions, you need to include the *eic.h* header file found in the *include* subdirectory of the EiC source directory, and link with the libraries *libeic* and *libstdClib* found in the *lib* subdirectory of the EiC source directory. To play around some more with the embedding capabilities of EiC, check *embedEiC.c* in the *main/example* subdirectory of the EiC source directory.

CINT is extensible through external functionalities coded in C and C++— this means you can create customised versions of CINT containing your extra functionality as an added feature. You could use this feature to interpret sources that use your external functionalities, without supplying the headers and sources of those. To embed these in CINT, you need *makecint*—an interpreter compiler that is built along with CINT. *Makecint* automates the process of embedding external functionalities coded in C and C++ implementations; it auto-generates the necessary wrapper code to add the extra functionalities to your customised CINT.

To further explore this powerful feature of CINT, copy the C++ header and source files *mycint.hpp* and mycint.cpp from the source's Zip to a temporary directory, and run the following command:

makecint -mk Makefile -o mycint -H mycint.hpp -C++ mycint.cpp

The *-mk* switch sets the name of the makefile generated by *makecint; -o* sets the name of the customised version of CINT, and *-H* and *-C++* specify the header and source file names, respectively. For more information, consult the man page.